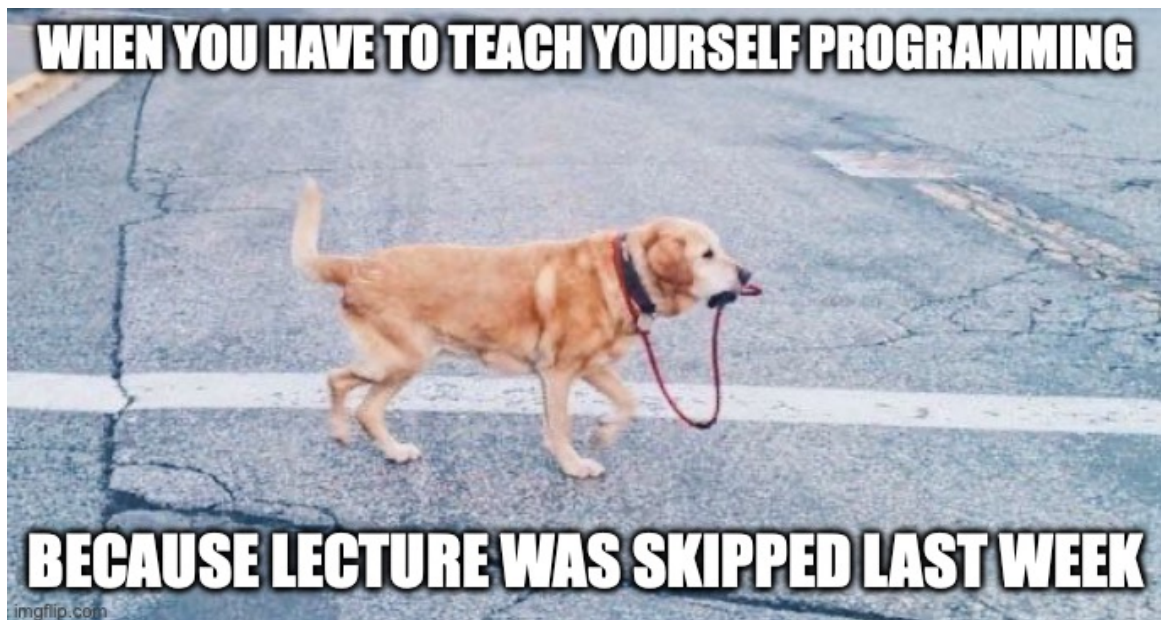


Übung 04.05. - 08.05.2026
Blatt 03

Besprechung der Aufgaben in der Woche vom 11.05.2026.



Im Grunde genommen haben wir eine große Aufgabe hergenommen und sie in drei logisch zusammenhängende und aufeinander aufbauende Teile portioniert. Zum Abarbeiten und zur Besprechung hielten wir es für sinnvoller, drei Teile (statt nur einer großen Aufgabe) daraus zu machen. Die Aufgaben können nicht getrennt voneinander kompiliert werden. Wir empfehlen, die Aufgaben 1, 2 und 3 zuerst durchzulesen und dann abzuarbeiten. Um Aufgabe 4 zu testen (ab (b)) ist die Lösung von den vorherigen Aufgaben sinnvoll.

Aufgabe 1 [**Locatable** - Interface selbst schreiben]

Erzeugen Sie ein Interface namens **Locatable**. Dieses besitzt die vier abstrakten Methoden **getNorth()**, **getSouth()**, **getEast()** und **getWest()**, welche alle ein **Cell**-Objekt zurückgeben.

Aufgabe 2 [**Cell**: Interface implementieren]

Erweitern Sie Ihr Programm aus Aufgabe 1 um die Klasse **Cell**. Als private Attribute besitzt **Cell** einen **boolean alive** und zwei **int**-Variablen **row** und **col**, welche die Zeile bzw. Spalte angeben, in der sich die Zelle befindet. Weiterhin gibt es die privaten vier **Cell**-Attribute **north**, **south**, **east** und **west**, welche jeweils den direkt angrenzenden Nachbarzellen in die jeweilige Richtung entsprechen. Schließlich besitzt **Cell** das private **Board**-Attribut **board** (siehe Aufgabe 3).

- (a) Erzeugen Sie einen Konstruktor, der **row**, **col** und **board** übergeben bekommt und entsprechend setzt.
- (b) Erzeugen Sie Getter und Setter für **alive**. Achten Sie auf die Konvention bei der Namensgebung.
- (c) Implementieren Sie das Interface **Locatable** (siehe Aufgabe 1), welches die Getter für die vier Nachbarzellen vorgibt. Anstelle von vier Settern schreiben Sie die Methode **setNeighbors(Cell north, Cell south, Cell east, Cell west)**, welche alle vier Nachbarzellen gleichzeitig setzt.
- (d) Schreiben Sie eine **printCell()** Methode, welche **"1"** zurückgibt, wenn die Zelle lebt. Sonst wird **"0"** zurückgegeben.

Aufgabe 3 [**Board**: Queue + LinkedList + ArrayDeque]

Erweitern Sie Ihr Programm um die Klasse **Board**. Als private Attribute besitzt die Klasse ein zweidimensionales **Cell**-Array namens **grid**, sowie zwei **int**-Variablen **rows** und **cols**, welche die Anzahl an Zeilen bzw. Spalten speichern, aus denen **grid** besteht.

- (a) Erzeugen Sie einen Konstruktor, der **rows** und **cols** übergeben bekommt und entsprechend setzt. Außerdem soll ein neues **grid** der Größe **rows** x **cols** erzeugt und entsprechend mit **Cell**-Objekten gefüllt werden. Setzen Sie abschließend im Konstruktor die Nachbarn jeder Zelle aus **grid**. Dabei gilt: Gibt es keinen Nachbarn in eine Richtung, weil

die Zelle am Rand oder in einer Ecke von **grid** liegt, so verweist die Zelle als Nachbarn in die entsprechende Richtung auf sich selbst.

- (b) Erzeugen Sie Getter für **grid**, **cols** und **rows**. Erweitern Sie anschließend die Klasse **Cell** um die **isCorner()** bzw. **isEdge()** Methode, welche jeweils einen **boolean** zurückliefern, der wiedergibt, ob die entsprechende Zelle in einer Ecke bzw. am Rand von **grid** liegt.
- (c) Erweitern Sie die Klasse **Board** um die private Methode **allAlive()**, welche einen **boolean** zurückgibt, der angibt, ob alle Zellen auf dem Brett am Leben sind.
- (d) Ergänzen Sie die Methode **printBoard()**, welche **grid** auf der Konsole zeigt. Überlegen Sie sich eine verständliche Darstellung.
- (e) Erweitern Sie die Klasse **Board** um die Methode **spreadLife(Queue<Cell> startCells)**, welche einen **int** zurückgibt. Die Methode simuliert die Ausbreitung von Leben auf dem Brett: Die übergebenen Startzellen werden zunächst zum Leben erweckt. Anschließend werden in jedem Schritt alle noch nicht lebendigen Nachbarn der aktuell lebendigen Zellen aktiviert. Die Methode gibt die Anzahl der Schritte zurück, die benötigt wurden, um alle Zellen auf dem Brett lebendig zu machen. Dabei ist der initiale Schritt, in dem die ersten, übergebenen Zellen aktiviert werden, als erster Schritt zu zählen.

Hinweise:

- Eine einmal zum Leben erweckte Zelle bleibt am Leben.
 - Verwenden Sie für diese Aufgabe das Interface **Queue<E>** aus **java.util**. Eine **Queue** ist eine Warteschlange nach dem *First-In-First-Out*-Prinzip (FIFO): Elemente werden hinten eingefügt (**add(e)**) und vorne entnommen (**poll()**). Sie eignet sich daher besonders gut für eine Breitensuche, bei der alle Nachbarn einer Ebene verarbeitet werden, bevor die nächste Ebene beginnt. Da **Queue<E>** ein Interface ist, benötigen Sie zur Instanziierung eine konkrete Implementierung. Verwenden Sie hierfür **new LinkedList<>()**, eine Ihnen noch unbekannte Klasse aus **java.util**, die unter anderem das **Queue**-Interface implementiert.
- (f) Ersetzen Sie in Ihrer **spreadLife** Implementierung die Verwendung von **LinkedList<>** durch **ArrayDeque<>**. Achten Sie darauf, dass Ihr Code ansonsten unverändert bleibt. Erklären Sie, warum der Austausch problemlos möglich ist. Welche Rolle spielt dabei das Interface **Queue<E>**?

Aufgabe 4 [ClassInfo: getClass + anonymes Objekt mit Methodenaufruf]

In Java ist jede Klasse zur Laufzeit durch ein Objekt vom Typ **Class<?>** repräsentiert. Dieses Objekt lässt sich über die Methode **getClass()** abrufen, die jedes Objekt in Java von der Klasse **Object** erbt (anders ausgedrückt: auch, wenn Sie **getClass** in Ihrer Klasse nicht implementiert sehen, existiert sie dort und kann über den Punktoperator aufgerufen werden). Über **Class<?>**-Objekte lassen sich zur Laufzeit Informationen über eine Klasse abfragen – dieses Konzept nennt sich *Reflection*.

- (a) Schreiben Sie eine Klasse **ClassInfo**, die ein beliebiges **Object obj** im Konstruktor entgegennimmt und als privates **Object**-Attribut speichert. Beachten Sie, dass in dieser Variablen alle möglichen Objekte gespeichert werden können (mehr dazu in naher Zukunft). Fügen Sie folgende Methoden hinzu, die jeweils einen **String** zurückgeben:

- **getClassName()**: Gibt den einfachen Namen der Klasse zurück (z. B. "**Cell**"). Verwenden Sie hierfür **getClass()** sowie **getSimpleName()**.
- **getFullName()**: Gibt den vollständigen Namen der Klasse inklusive Paketpfad zurück (z. B. "**game.Cell**"). Verwenden Sie hierfür **getName()**.
- **getPackageName()**: Gibt den Namen des Pakets zurück, in dem sich die Klasse befindet (z. B. "**game**"). Verwenden Sie hierfür **getClass().getPackage().getName()**.

Hinweis: **getPackage()** kann **null** zurückgeben, wenn die Klasse in keinem benannten Paket liegt. Behandeln Sie diesen Fall in Ihrer Implementierung.

- (b) Testen Sie Ihre Klasse **ClassInfo**, indem Sie Objekte verschiedener Typen übergeben, z. B. ein **Cell**-Objekt, ein **Board**-Objekt sowie ein Objekt einer Klasse aus **java.util** (z. B. **new LinkedList<>()**). Notieren Sie jeweils die Ausgaben der drei Methoden und erklären Sie die Unterschiede.
- (c) Erweitern Sie **ClassInfo** um die Methode **isInSamePackage(Object other)**, die einen **boolean**-Wert zurückgibt. Die Methode soll prüfen, ob sich das gespeicherte Objekt und **other** im selben Paket befinden. Überlegen Sie sich dabei, welche der Methoden aus Aufgabenteil (a) Sie wiederverwenden können.

Hinweis: Vergleichen Sie die beiden **Strings**, die den Packagenamen beinhalten, mit der Methode **packageName1.equals(packageName2)**.

- (d) In Java ist es möglich, ein Objekt zu erzeugen und direkt eine Methode darauf aufzurufen, ohne es in einer Variable zu speichern. Im Prinzip lautet der Befehl dann so:

new Klassenname(argumente).methode()

Dies empfiehlt sich, wenn ein Objekt nur einmalig und ausschließlich für diesen einen Aufruf benötigt wird. Eine Variable wäre in diesem Fall irreführend, da sie den Eindruck erweckt, das Objekt werde noch einmal verwendet. Betrachten Sie Ihre Implementierung von **isInSamePackage(Object other)**. Das dort erzeugte **ClassInfo**-Objekt für **other** wird genau einmal verwendet. Schreiben Sie die Methode so um, dass kein benanntes Zwischenobjekt mehr existiert.